

AI-Powered Cloud ERP Suite

A multi-tenant ERP platform for finance, HR & payroll, inventory, projects, AI forecasting, anomaly detection, and intelligent approvals.

Reference

week4-devops branch

Prepared

2026-07-09

Scope of this document

Describes the system as it actually runs today: architecture, functional coverage, security posture, deployment shape, and known gaps against a broader enterprise ERP specification. Written from the running code and a live local verification pass, not from aspirational requirements.

SECTION 1

Executive Summary

This is a working multi-tenant ERP backend (Express + TypeScript + MongoDB) with a React/Vite frontend, covering HR, Payroll, Finance (AP/AR + vendor 3-way matching), Inventory & Procurement, Project Management, AI Demand Forecasting (Prophet + LSTM), statistical Anomaly Detection, risk-scored Intelligent Approvals, a BI Dashboard with live SSE updates, a Notification engine, TOTP-based two-factor authentication, and a tenant-wide, hash-chained Audit Log with GDPR self-service.

The system is deliberately built on a leaner stack than a full enterprise reference architecture would specify (see Section 3) — MongoDB instead of PostgreSQL, JWT + TOTP instead of a federated OIDC identity provider, a Vite SPA instead of Next.js. These are documented, reasoned trade-offs (see docs/adr/), not gaps introduced by oversight.

What was verified live, this pass

- Login !' JWT issuance !' role-scoped access (ADMIN/HR/EMPLOYEE)
- TOTP MFA: setup !' confirm !' login now requires a second factor !' verified code exchanges for a session token
- Defense in depth: a pre-MFA token is rejected by authMiddleware even if captured before the second factor is provided
- Dashboard KPIs against seeded finance/inventory/payroll data (revenue, expenses, inventory value, payroll totals)
- Anomaly detection: real z-score scan against vendor-payment and goods-receipt history, flags created and reviewed
- Approvals: risk-scored requests routed to auto-approve vs. pending, approved/rejected through the real decision endpoint
- Audit log: mutation entries written automatically, hash-chain verification endpoint confirmed valid

Defects found and fixed during this pass

- server.ts was missing its startServer() call entirely (dropped in an earlier commit) — the API never bound to a port
- Every dashboard KPI aggregation matched tenantId as a string against an ObjectId-typed field inside \$match, silently returning zero for any real user
- config/redis.ts returned null whenever NODE_ENV=production, which would have silently disabled every BullMQ queue (payroll, email, webhook, reorder) on any real deployment — fixed to prefer a REDIS_URL connection string

SECTION 2

Architecture Overview

Deployment topology (as configured in docker-compose.yml / render.yaml / the Helm chart):

```
Vite/React SPA (nginx locally, static host on Render)
  |
  v /api (dev proxy) or VITE_API_BASE_URL (prod)
Express + TypeScript API <---> Python FastAPI ML service (Prophet + LSTM)
  |           |
  v           v
MongoDB Atlas Redis (BullMQ queues, rate limiting, KPI cache)
  |
  v
Prometheus (/metrics) -> Grafana / monitoring/prometheus.yml
```

Design patterns actually in use

- Tenant scoping: every controller filters by tenantId from the decoded JWT — verified consistently across HR/Finance/Inventory/Projects
- Purpose-scoped tokens: a login requiring MFA gets a 5-minute mfa_pending JWT, structurally incapable of reaching any real protected route
 - Queue-backed side effects: payroll runs, email/webhook delivery, reorder alerts, and forecast retraining are BullMQ jobs, not inline request handling
 - Event bus: EventEmitter2 (config/eventBus.ts) decouples anomaly/approval detection from notification delivery
 - Append-only audit trail: a single global middleware (auditLogger) hooks res.on('finish') rather than instrumenting each controller individually
 - Soft-delete / anonymize over hard-delete: GDPR erasure anonymizes the User document rather than removing it, preserving referential integrity for payroll/approvals/audit history

SECTION 3

Functional Coverage (F-01 – F-12)

Verdicts reflect code actually present and, where noted, exercised live — not aspirational claims.

ID	REQUIREMENT	STATUS	EVIDENCE
F-01	Multi-tenant auth (SSO)	Partial	Real tenant isolation + TOTP MFA verified live. Still no SAML/OIDC federation to an external IdP.
F-02	Financial ledger (GL)	Partial	Ledger, FX, and period-lock routes exist. Double-entry balance enforcement not verified.
F-03	AP/AR automation	Partial	Invoice + 3-way match pipeline exists. No OCR anywhere in the codebase.
F-04	HR & payroll engine	Mostly met	Employee/Payroll/Attendance/Leave + payslip PDF. KPI math verified correct live.
F-05	Supply chain & inventory	Met	PO -> goods receipt -> inventory flow with a dedicated reorder-automation worker.
F-06	AI demand forecasting	Met	Both Prophet and LSTM models ship, plus a model_registry.py for versioning.
F-07	Project management	Met	Full CRUD backend and a frontend screen (Projects/Tasks/Milestones).
F-08	Business intelligence	Partial	KPIs, widgets, live SSE, PDF/CSV export all functional. No drag-and-drop builder.
F-09	Audit & compliance log	Met	Tenant-wide hash-chained mutation log + GDPR export/erase.
F-10	Notification engine	Mostly met	In-app + email + HMAC-signed webhook, per-user preferences. No SMS channel.
F-11	API gateway & webhooks	Partial	Full REST surface + OpenAPI 3.1 spec at /api-docs. No GraphQL.
F-12	Offline / PWA support	Met	Manifest + service worker (app shell + cached API reads).

SECTION 4

Technology Stack

LAYER	ACTUAL
Frontend	Vite 6 + React 18, hand-rolled CSS components, React Router
Backend	Node 22 + Express 5 + TypeScript
Database	MongoDB Atlas + Mongoose (local MongoDB fallback documented in docs/adr/0001)
Cache / Queues	Redis 7 + BullMQ (payroll, email, webhook, reorder, forecasting, reports)
AI / ML	Python FastAPI + Prophet + LSTM (PyTorch), file-based model registry
Auth	JWT (HS256) + bcrypt + TOTP 2FA — see docs/adr/0002 for the SSO trade-off
Containers	Multi-stage Docker builds, non-root user, Alpine base
Orchestration	Kubernetes manifests + Helm chart, or render.yaml for a single-command Render deploy
CI/CD	GitHub Actions — lint, build, test (Jest + pytest), Docker image builds
Security scanning	TruffleHog (secrets), Trivy (containers), Snyk (deps, conditional on a token)
Observability	prom-client + /metrics, monitoring/prometheus.yml scrape config
API docs	OpenAPI 3.1 at /api/openapi.json, Redoc UI at /api-docs

Security & Compliance

Controls in place

- TOTP two-factor authentication (RFC 6238, implemented against Node's crypto — no external dependency); purpose-scoped pre-auth tokens can't reach protected routes
- Helmet: strict CSP (default-src 'none'), HSTS, frameguard, referrer-policy, permitted-cross-domain-policies
- CORS allowlist via ALLOWED_ORIGINS — see SECURITY.md for why CSRF protection is a non-issue for bearer-token auth
- Rate limiting: in-memory sliding window on general + auth-specific routes
- Mass-assignment guard: tenantId is always applied after the request-body spread on create; WHERE clauses on update always include tenantId
- Dependency/secret/container scanning wired into CI (TruffleHog, Trivy, Snyk)
- Tenant-wide, hash-chained, append-only audit log with a chain-verification endpoint
- GDPR self-service export (Art. 15/20) and erasure (Art. 17), implemented as anonymization rather than hard delete

Known gaps

- No SSO/OIDC/SAML federation to an external identity provider (docs/adr/0002) — MFA is covered, federation is not
- No first-user bootstrap flow — the first tenant + ADMIN user must be seeded directly against the database (see docs/deploy/render.md, step 6, for the deployed-environment version of this)
- No cross-collection transactions around money movement (invoice payment + ledger entry are sequential writes, not atomic)

Deployment, Testing & Operations

Deployment

`docker-compose.yml` runs `api`, `ml-service`, `web`, `redis`, and `postgres` (unused by the app; present for future use) as a local stack. `render.yaml` provisions `erp-api`, `erp-redis`, and `erp-web` as a Render Blueprint — see `docs/deploy/render.md` for the two manual steps (MongoDB URI, and wiring the two services' generated URLs together) Render can't fill in automatically. Kubernetes manifests (`k8s/`) and a Helm chart (`helm/`) describe a cluster deployment for larger scale.

Testing

Jest + Supertest + `mongodb-memory-server` cover auth, anomaly detection, and approvals on the API side; `pytest` covers both forecasting models and the FastAPI service. No end-to-end (Playwright) or load-testing (`k6`) suite exists yet.

Observability

Prometheus scrapes `/metrics` on the API (`prom-client`). No Grafana dashboards, log aggregation (Loki), or distributed tracing (OpenTelemetry) are configured.

Roadmap — Highest-Leverage Next Steps

- 1. Bootstrap flow: an environment-gated "create first tenant + admin" endpoint, removing the manual-seed requirement in both local dev and the Render deploy
- 2. Wrap AP/AR payment and payroll-run writes in a Mongo session/transaction for atomicity
- 3. Add Playwright (e2e) and k6 (load) suites — currently the only tested paths are unit/integration
- 4. Re-enable Bull Board behind roleMiddleware(["ADMIN"]) for queue/dead-letter visibility
- 5. If enterprise SSO becomes a real requirement: introduce Keycloak or an OIDC provider behind the existing tenant model, layering it alongside the TOTP MFA already in place rather than replacing it
- 6. Add the ml-service (Prophet/LSTM) as a fourth deployed service once the frontend actually calls it — currently backend-only, so it was left out of render.yaml